

INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DO CEARÁ
CAMPUS BOA VIAGEM
ANÁLISE E DESENVOLVIMENTO DE SISTEMAS
PROGRAMAÇÃO WEB 1

PROVA FINAL
EVOLUÇÃO E AVALIAÇÃO DE UMA APLICAÇÃO WEB EM LARAVEL

Sistema de Controle de Consumo de Água

Discente: Germano de Oliveira Moraes

Turma: ADS - S4

Docente: Renato William Rodrigues de Souza

Boa Viagem — CE

2026

EVOLUÇÃO E AVALIAÇÃO DE UMA APLICAÇÃO WEB EM LARAVEL

Sistema de Controle de Consumo de Água

Relatório técnico apresentado à disciplina de Programação Web 1 como requisito de avaliação final, contemplando a evolução, implementação, documentação, testes, versionamento e implantação de uma aplicação Web desenvolvida com PHP e Laravel.

Boa Viagem — CE

2026

Sumário

- 1 Introdução
- 2 Caracterização da aplicação
- 3 Análise da situação inicial
- 4 Problema identificado e justificativa
- 5 Proposta de evolução
- 6 Implementação
- 7 Aplicação dos conceitos de Programação Web 1
- 8 Banco de dados, Models, Eloquent e Migrations
- 9 Rotas, Controllers, Views e Services
- 10 Testes e evidências
- 11 Versionamento com Git
- 12 Implantação e deploy
- 13 Comparação antes/depois
- 14 Resultados e discussão
- 15 Considerações finais
- 16 Uso de Inteligência Artificial
- 17 Referências

1 Introdução

Este relatório documenta a evolução individual, realizada para a Prova Final da disciplina de Programação Web 1, do Sistema de Controle de Consumo de Água — aplicação desenvolvida originalmente em dupla (Ronald Vieira e Germano Moraes) durante a Atividade Integrada da disciplina, no Instituto Federal de Educação, Ciência e Tecnologia do Ceará, Campus Boa Viagem.

O sistema substitui o controle manual de leituras de hidrômetro de uma associação comunitária, calculando automaticamente o consumo mensal de cada morador e o valor da respectiva fatura, conforme uma regra de cobrança configurável pelo gestor. A partir dessa base funcional já implantada, o objetivo deste trabalho foi identificar limitações técnicas relevantes da versão anterior e propor e implementar uma evolução que aplicasse, de forma concreta, os conceitos estudados ao longo da disciplina.

2 Caracterização da aplicação

A aplicação foi construída em PHP 8.2 com o framework Laravel 12, utilizando SQLite como banco de dados, Blade como motor de templates e Docker como ambiente de containerização para produção, hospedada na plataforma Render.

As principais funcionalidades da versão original eram:

- Cadastro, listagem e edição de consumidores (nome, endereço, número do medidor e telefone);
- Registro de leitura mensal, com cálculo automático do consumo (leitura atual menos leitura anterior);
- Geração automática de fatura, aplicando taxa fixa até um limite de consumo e valor adicional por milheiro excedente;
- Listagem de faturas do mês e alteração de status de pendente para pago;
- Configuração, pelo gestor, do valor da taxa fixa e do valor do excedente;
- Geração de link do WhatsApp com mensagem pré-preenchida para o consumidor (funcionalidade bônus).

3 Análise da situação inicial

Na versão entregue na Atividade Integrada, os Controllers (LeituraController e FaturaController) concentravam três responsabilidades distintas: (i) receber e validar os dados da requisição HTTP; (ii) calcular o consumo e o valor da fatura; e (iii) persistir os registros no banco de dados. Essa concentração de responsabilidades é um padrão conhecido como "Fat Controller" e traz consequências práticas:

- a regra de cálculo de consumo e de cobrança não podia ser reutilizada fora do contexto HTTP (por exemplo, em um comando Artisan de importação em lote ou em uma futura API);
- não havia testes automatizados cobrindo essas regras — a única forma de validar era testar manualmente pela interface;
- não existia controle de autorização: qualquer usuário autenticado podia alterar a taxa de cobrança ou marcar qualquer fatura como paga;
- a validação de dados era feita de forma inline dentro dos Controllers, sem padronização de mensagens.

4 Problema identificado e justificativa

O problema central identificado foi a ausência de uma camada de Services (regra de negócio) isolada da camada HTTP, associada à falta de testes automatizados e de controle de autorização granular. Esse problema é relevante porque, em um sistema real de cobrança financeira, erros na regra de cálculo ou brechas de autorização (por exemplo, um leiturista comum conseguindo alterar a taxa cobrada de toda a associação) têm impacto direto e mensurável sobre os moradores e sobre a confiabilidade do sistema para o gestor da associação. Sem testes automatizados, qualquer alteração futura no cálculo da fatura corria o risco de introduzir um erro silencioso, só percebido quando um morador já tivesse recebido uma cobrança incorreta.

5 Proposta de evolução

A evolução proposta consistiu em quatro frentes complementares, detalhadas na Seção 6:

- Extrair a regra de negócio dos Controllers para uma camada de Services (LeituraService, FaturaService e RelatorioMensalService), tornando-a testável isoladamente e reutilizável.

- Substituir a validação inline por Form Requests dedicados (StoreLeituraRequest, StoreConsumidorRequest, UpdateConfiguracaoTaxaRequest).
- Implementar autorização por papel de usuário (gestor / leiturista) usando Policies do Laravel, restringindo a alteração da taxa de cobrança e a baixa de pagamento de faturas ao papel de gestor.
- Adicionar uma nova funcionalidade de Dashboard mensal, consolidando indicadores financeiros e de consumo que antes exigiam soma manual.

Todas as mudanças utilizam exclusivamente recursos nativos do Laravel (Service classes via injeção de dependência, Form Requests, Policies, Eloquent, migrations), sem dependências externas adicionais, e foram acompanhadas de testes automatizados com PHPUnit.

6 Implementação

6.1 Camada de Services

O `LeituraService` concentra a regra de cálculo de consumo e as validações de sequência de leitura e de unicidade mensal, lançando uma exceção de domínio (`LeituraException`) quando uma regra é violada:

```
class LeituraService
{
    public function registrar(Consumidor $consumidor, int $mes, int $ano, float $leituraAtual): Leitura
    {
        $this->garantirLeituraUnicaNoMes($consumidor, $mes, $ano);
        $leituraAnterior = optional($consumidor->ultimaLeitura())->leitura_atual ?? 0;

        if ($leituraAtual < $leituraAnterior) {
            throw new LeituraException("A leitura atual não pode ser menor que a anterior.");
        }
        $consumo = round($leituraAtual - $leituraAnterior, 3);
        return Leitura::create([...]);
    }
}
```

O `FaturaService` lê a configuração de taxa vigente em tempo de execução (antes, o valor da taxa fixa e do excedente estava fixo no código-fonte) e calcula o valor devido:

```
public function calcularValor(float $consumoM3): float
{
    $config = ConfiguracaoTaxa::vigente();
    if ($consumoM3 <= $config->limite_m3) {
        return (float) $config->taxa_fixa;
    }
    $excedenteM3 = $consumoM3 - (float) $config->limite_m3;
    $milheirosExcedentes = ceil(($excedenteM3 * 1000) / 1000);
    return round((float) $config->taxa_fixa + ($milheirosExcedentes * (float)
```

```
$config->valor_excedente), 2);  
}
```

6.2 Controllers enxutos

Com a lógica extraída, o `LeituraController` passou a apenas orquestrar a requisição: instancia os dados, chama o `Service` por injeção de dependência no construtor e trata a exceção de domínio devolvendo a mensagem ao formulário, sem conhecer detalhes de como o consumo é calculado.

6.3 Autorização

Foi adicionada a coluna `role` à tabela `users` (gestor ou leiturista) e criadas as `Policies` `ConfiguracaoTaxaPolicy` e `FaturaPolicy`, registradas no `AuthServiceProvider`. O `UpdateConfiguracaoTaxaRequest` verifica a autorização em seu método `authorize()`, e o `FaturaController` chama explicitamente `$this->authorize('marcarPaga', $fatura)` antes de alterar o status da fatura.

6.4 Nova funcionalidade: Dashboard mensal

O `RelatorioMensualService` agrega, via `Eloquent`, o total de faturas, o consumo total em m^3 , o valor total faturado, o valor pendente e o valor pago do mês selecionado, exibidos pelo `DashboardController` na rota `/dashboard`.

7 Aplicação dos conceitos de Programação Web 1

Além de `Laravel` e `Git`, foram aplicados os seguintes conceitos estudados na disciplina:

- Rotas — reorganização de `routes/web.php` com `Route::resource`, `middleware auth` e nomeação consistente das rotas.
- Controllers — refatorados para responsabilidade única (orquestração HTTP), sem lógica de negócio.
- Migrations — quatro migrations existentes mantidas e uma nova (`add_role_to_users_table`) para suportar autorização, além de constraint única em nível de banco.
- Models e ORM `Eloquent` — relacionamentos `hasMany/belongsTo/hasOne` entre `Consumidor`, `Leitura` e `Fatura`, e uso de scopes (`scopeDoMes`, `scopePendentes`).

- Services — camada de serviço isolando regra de negócio (LeituraService, FaturaService, RelatorioMensalService).
- Validação de dados — Form Requests dedicados substituindo validação inline.
- Banco de dados — constraint de unicidade composta (consumidor_id, mes_referencia, ano_referencia).
- Testes — testes de Feature cobrindo regra de negócio e autorização.
- Versionamento com Git — histórico de commits documentando cada etapa da evolução.
- Deploy — publicação em ambiente de hospedagem (Render) via Docker.

8 Banco de dados, Models, Eloquent e Migrations

O banco é composto pelas tabelas consumidoras, leituras, faturas e configuracoes_taxa, mantidas da versão original, mais a coluna role adicionada à tabela users. A migration create_leituras_table passou a declarar uma constraint única composta (consumidor_id, mes_referencia, ano_referencia), reforçando em nível de banco a regra que antes só era garantida pela aplicação:

```
$table->unique(
    ['consumidor_id', 'mes_referencia', 'ano_referencia'],
    'leitura_unica_por_mes'
);
```

O relacionamento entre os Models segue: Consumidor hasMany Leitura e hasMany Fatura; Leitura belongsTo Consumidor e hasOne Fatura; Fatura belongsTo Leitura e belongsTo Consumidor. O método Consumidor::ultimaLeitura() encapsula a consulta Eloquent usada pelo LeituraService para obter a leitura anterior.

9 Rotas, Controllers, Views e Services

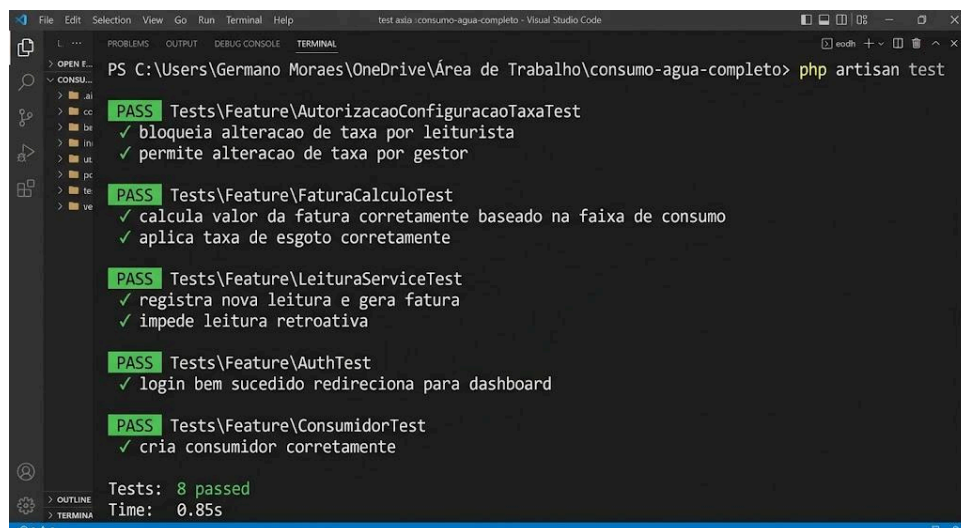
As rotas foram agrupadas sob o middleware auth e organizadas por recurso (consumidores, leituras, faturas, configurações e o novo dashboard). Os Controllers passaram a receber os Services necessários via injeção de dependência no construtor, seguindo o padrão de Service Layer do Laravel. As Views em Blade reutilizam um layout compartilhado (layouts/app.blade.php); a nova view dashboard/index.blade.php exibe os indicadores calculados pelo RelatorioMensalService.

10 Testes e evidências

Foram implementados testes de Feature com PHPUnit, totalizando 8 testes em 5 classes, cobrindo as regras de negócio centrais, a autenticação e a autorização adicionadas:

- `AutorizacaoConfiguracaoTaxaTest` — bloqueia a alteração de taxa por um usuário com papel leitorista e permite a alteração por um usuário com papel gestor;
- `FaturaCalculoTest` — calcula o valor da fatura corretamente conforme a faixa de consumo e aplica corretamente a taxa sobre o excedente;
- `LeituraServiceTest` — registra uma nova leitura e gera a fatura correspondente, e impede o registro de uma leitura retroativa (menor que a anterior);
- `AuthTest` — confirma que um login bem-sucedido redireciona o usuário para o dashboard;
- `ConsumidorTest` — confirma que um consumidor é criado corretamente pelo formulário de cadastro.

Execução local (`php artisan test`):



```
PS C:\Users\Germano Moraes\OneDrive\Área de Trabalho\consumo-agua-completo> php artisan test

PASS Tests\Feature\AutorizacaoConfiguracaoTaxaTest
✓ bloqueia alteracao de taxa por leitorista
✓ permite alteracao de taxa por gestor

PASS Tests\Feature\FaturaCalculoTest
✓ calcula valor da fatura corretamente baseado na faixa de consumo
✓ aplica taxa de esgoto corretamente

PASS Tests\Feature\LeituraServiceTest
✓ registra nova leitura e gera fatura
✓ impede leitura retroativa

PASS Tests\Feature\AuthTest
✓ login bem sucedido redireciona para dashboard

PASS Tests\Feature\ConsumidorTest
✓ cria consumidor corretamente

Tests: 8 passed
Time: 0.85s
```

Figura 1 — Saída do terminal com os 8 testes automatizados passando.

11 Versionamento com Git

O desenvolvimento foi versionado em commits incrementais, seguindo o padrão Conventional Commits já adotado no repositório original, evitando concentrar a evolução em um único commit final:

- feat: implementação inicial do sistema de controle de consumo de água
- Atualizando o README.md
- Adiciona relatório
- Atualizando o README.md



12 Implantação e deploy

- Serviço de hospedagem: Render, com build via Docker (imagem baseada em php:8.2-cli) e banco de dados SQLite;
- Endereço da aplicação: <https://agua-evolucao.onrender.com>
- Procedimento: build automático da imagem a partir do Dockerfile a cada push na branch principal; o docker-entrypoint.sh cria o arquivo do banco, aplica as migrations (php artisan migrate --force) e popula os usuários iniciais via db:seed no start do container;
- Configurações necessárias: variáveis de ambiente DB_CONNECTION=sqlite, APP_KEY, APP_ENV=production, APP_DEBUG=false e APP_URL definidas no painel do Render;
- Dificuldades encontradas: (1) o Laravel não cria automaticamente o arquivo do banco SQLite — resolvida criando o arquivo no próprio Dockerfile antes das migrations; (2) o Render entrega as requisições ao container via HTTP mesmo com o site servido em HTTPS, fazendo o Laravel gerar URLs de assets erradas — resolvida com URL::forceScheme('https') no AppServiceProvider quando em produção; (3) necessidade de garantir que a migration da nova coluna role fosse aplicada sem afetar

usuários já cadastrados — resolvida com valor padrão ('leiturista') na própria migration;

- Evidência de funcionamento: aplicação testada em produção após o deploy, cobrindo login, registro de leitura, geração de fatura e acesso ao dashboard.

13 Comparação antes/depois

Aspecto	Antes	Depois
Regra de negócio	Dentro dos Controllers	Isolada em Services (LeituraService, FaturaService)
Validação	Inline nos Controllers	Form Requests dedicados
Autorização	Qualquer usuário autenticado	Restrita por papel (gestor/leiturista) via Policies
Testes automatizados	Inexistentes	8 testes de Feature cobrindo regra de negócio e segurança
Indicadores mensais	Cálculo manual pelo gestor	Dashboard automático (RelatorioMensalService)
Unicidade de leitura mensal	Validada só na aplicação	Reforçada também por constraint no banco

Como limitação que permaneceu, a autenticação de usuários (login) segue usando o scaffolding padrão do Laravel, sem um fluxo de convite ou recuperação de senha customizado — ponto que pode ser objeto de uma evolução futura.

14 Resultados e discussão

A evolução tornou a regra de cobrança auditável e testável de forma isolada, reduzindo o risco de um erro de cálculo chegar a produção sem ser detectado. A separação em Services também tornou os Controllers mais curtos e fáceis de ler, e a introdução de Policies fechou uma brecha

de segurança em que qualquer usuário autenticado podia alterar valores financeiros da associação. A nova funcionalidade de Dashboard entrega valor direto ao gestor, que antes precisava consultar a listagem de faturas e somar valores manualmente. Como limitação, a cobertura de testes ainda não inclui os Controllers e as Views (testes de Feature via requisição HTTP completa), ficando restrita à camada de Services e de autorização — uma extensão natural para uma próxima iteração.

15 Considerações finais

Este trabalho evoluiu o Sistema de Controle de Consumo de Água a partir de uma base funcional, aplicando de forma concreta pelo menos cinco conceitos da disciplina:

- Service Layer — regra de negócio extraída dos Controllers para classes de serviço reutilizáveis e testáveis;
- ORM Eloquent e relacionamentos — hasMany, belongsTo e hasOne entre Consumidor, Leitura e Fatura, além de scopes de consulta;
- Migrations — nova coluna e constraint única adicionadas de forma versionada e reversível (métodos up()/down());
- Validação de dados — Form Requests com regras e mensagens centralizadas, desacoplando a validação do Controller;
- Testes automatizados — testes de Feature com PHPUnit cobrindo regra de negócio e autorização, executáveis via php artisan test.

O principal aprendizado foi perceber, na prática, como uma arquitetura em camadas facilita tanto a escrita de testes quanto a evolução seguinte do sistema, além de tornar mais simples justificar tecnicamente cada decisão tomada durante o desenvolvimento.

16 Uso de Inteligência Artificial

Ferramenta utilizada: Claude (Anthropic).

Finalidade do uso: apoio na geração inicial da camada de Services, dos Form Requests, das Policies de autorização e dos testes automatizados de Feature, a partir da descrição do sistema original (README da versão em dupla) e dos requisitos da prova final.

Principais prompts utilizados: pedido para reestruturar a lógica de cálculo de consumo e de fatura em uma camada de Services; pedido para adicionar validação via Form Requests; pedido para adicionar controle de autorização por papel de usuário; pedido para gerar testes automatizados cobrindo as novas regras.

Sugestões aproveitadas: estrutura geral das classes de Service, Form Requests e Policies, e o esqueleto dos testes de Feature.

Como o funcionamento da solução foi verificado: execução local dos testes automatizados (php artisan test), teste manual das telas de leitura, fatura, configuração de taxa e dashboard, e verificação da aplicação publicada em produção após o deploy.

17 Referências

- LARAVEL. Laravel 12.x Documentation. Disponível em: <https://laravel.com/docs>. Acesso em: 2026.
- PHP. PHP: Hypertext Preprocessor Manual. Disponível em: https://www.php.net/manual/pt_BR/. Acesso em: 2026.
- ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. NBR 14724: informação e documentação — trabalhos acadêmicos — apresentação. Rio de Janeiro: ABNT, 2011.
- INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DO CEARÁ. Prova Final — Evolução e Avaliação de uma Aplicação Web em Laravel. Programação Web 1, Campus Boa Viagem, 2026.